Table of Contents

(Code snippets may be broken)

Introduction

Congratulations on your purchase of the Commodiesel 75 Personal Computer.
Designed for hobbyists and Lua programmers of varying skill levels, the Commodiesel 75 is ideal for creating games, controlling components, and automating tasks.

Your Commodiesel 75 is bundled with everything you need to begin computing immediately.

- PC Monitor
- PC Speaker
- 10 GPIO Expansions
- 2 Computer Joysticks

Commodiesel specs:

- 256 colours, sprite based graphics
- 16kb of video memory
- 8 CPU threads
- Graphical resolution of 100x82
- Text resolution of 300x246
- A 5-channel sound chip, with one sample channel

To run code, you must first create a .lua file that contains your code.

Example: `make test.lua -c print("test")`

Then, execute your code by running the file.

Example: `run test.lua`

It is recommended that you have baseline knowledge of the Lua programming language before continuing.

Lua

Code in the Commodiesel runs in a modified Lua sandbox environment, featuring added libraries and functionality designed to make it as simple as possible to interface with the computer hardware.

If you do not know how to run code, please read the Introduction page. (2)

This manual does not intend to teach you how to write in the Lua programming language and primarily serves as documentation on what has been added.

Loops can only run 120 times a second and may be affected by the servers TPS. Nested loops may also cause slowdown.

Globals

These are functions and variables which are accessible anywhere within your program.

`stop()`

This will immediately stop your program in its tracks. Identical to the Lua stop command.

`beep(hz, duration)`

Uses the Beeper inside of the computer to generate a tone for a set duration.

`arguments`

This is a numbered table containing extra arguments provided after the file name in the run command.

Modules

Module scripts are separate Lua files that packages code into tables for use with other programs. They can either have .lua or .mod file extensions.

Example module “mod.lua”

```lua
local module = {}
module.hi = "hello"
function module.testfunc(str)
    print(str)
end
return module
```

Example usage:

```lua
local module = require("mod.lua")
print(module.hi)
module.testfunc("world")
```

Drawing Colours

There is a total of 256 colours that can be displayed. Below is the colour palette formatted into the 16x16 table. This is ideal for representing the colours in hexadecimal. For example, 0x20 =32 = red, 0x2c = 44 = dark red.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	Clear	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D	0x0E	0x0F
1	0x10	0x11	0x12	0x13	0x14	0x15	0x16	0x17	0x18	0x19	0x1A	0x1B	0x1C	0x1D	0x1E	0x1F
2	0x20	0x21	0x22	0x23	0x24	0x25	0x26	0x27	0x28	0x29	0x2A	0x2B	0x2C	0x2D	0x2E	0x2F
3	0x30	0x31	0x32	0x33	0x34	0x35	0x36	0x37	0x38	0x39	0x3A	0x3B	0x3C	0x3D	0x3E	0x3F
4	0x40	0x41	0x42	0x43	0x44	0x45	0x46	0x47	0x48	0x49	0x4A	0x4B	0x4C	0x4D	0x4E	0x4F
5	0x50	0x51	0x52	0x53	0x54	0x55	0x56	0x57	0x58	0x59	0x5A	0x5B	0x5C	0x5D	0x5E	0x5F
6	0x60	0x61	0x62	0x63	0x64	0x65	0x66	0x67	0x68	0x69	0x6A	0x6B	0x6C	0x6D	0x6E	0x6F
7	0x70	0x71	0x72	0x73	0x74	0x75	0x76	0x77	0x78	0x79	0x7A	0x7B	0x7C	0x7D	0x7E	0x7F
8	0x80	0x81	0x82	0x83	0x84	0x85	0x86	0x87	0x88	0x89	0x8A	0x8B	0x8C	0x8D	0x8E	0x8F
9	0x90	0x91	0x92	0x93	0x94	0x95	0x96	0x97	0x98	0x99	0x9A	0x9B	0x9C	0x9D	0x9E	0x9F
a	0xA0	0xA1	0xA2	0xA3	0xA4	0xA5	0xA6	0xA7	0xA8	0xA9	0xAA	0xAB	0xAC	0xAD	0xAE	0xAF
b	0xB0	0xB1	0xB2	0xB3	0xB4	0xB5	0xB6	0xB7	0xB8	0xB9	0xBA	0xBB	0xBC	0xBD	0xBE	0xBF
c	0xC0	0xC1	0xC2	0xC3	0xC4	0xC5	0xC6	0xC7	0xC8	0xC9	0xCA	0xCB	0xCC	0xCD	0xCE	0xCF
d	0xD0	0xD1	0xD2	0xD3	0xD4	0xD5	0xD6	0xD7	0xD8	0xD9	0xDA	0xDB	0xDC	0xDD	0xDE	0xDF
e	0xE0	0xE1	0xE2	0xE3	0xE4	0xE5	0xE6	0xE7	0xE8	0xE9	0xEA	0xEB	0xEC	0xED	0xEE	0xEF
f	0xF0	0xF1	0xF2	0xF3	0xF4	0xF5	0xF6	0xF7	0xF8	0xF9	0xFA	0xFB	0xFC	0xFD	0xFE	0xFF

Before creating graphics, you may want to turn off the text rendering:

```
render.text = false
```

You can now test colours by changing the graphics background.

```
render.graphicsbg = 0xa8
```

To convert a colour from the palette to a Color3 value use:

```
color3.fromPalette(colour)
```

Textures

Textures store an array of pixels; textures can be assigned to sprites to display them on screen. Textures have a maximum size of 8x128.

To create a texture, use:

```
texture.new(size:vector2,pixeldata {number})
```

Example usage:

```
local C = 0x0f
local t = texture.new(vector2.new(4,4),{
C,0,0,C,
0,0,0,0
C,0,0,C
0,C,C,0,
})
```

Methods

`texture:SetPixels(pixeldata:{number}, offset: number?)`

Updates pixels at runtime.

```
texture.new(vector2.new(6,1))
wait(1)
t.SetPixels({0x20, 0x40, 0x60, 0x80, 0xa0, 0xc0})
```

`texture:Destroy()`

Deletes a texture from the texture cache. Do not call this if there are sprites on screen that are using the texture.

Sprites

Sprites display textures on screen. Sprites have a max size of 8x82. To create a sprite, use:

```
sprite.new(texture, size: Vector2, position: Vector2)
```

Example usage:

```
render.text = false
local C = 0x70
local t = texture.new(vector2.new(4,4),{
C,0,0,C,
0,0,0,0,
C,0,0,C,
0,C,C,0,
})
local s = sprite.new(t, vector2.new(4,4))
s.scale = vector2.new(5,5)
s.position = vector2.new(40,31)
```

Methods

`sprite:Destroy()`

`sprite:Clone()`

Properties

sprite.position: vector2

sprite.scale: vector2

sprite.offset: vector2

(Sprite sheets can be implemented through this by packing multiple frames onto a single texture and changing the offset using this property)

sprite.scaletype:Enum.ScaleType.Stretch or Enum.ScaleType.Tile

(1= stretch, 2 = tile)

sprite.tilescale: vector2

sprite.zindex: number (-128 to 128)

sprite.size: readonly vector2

sprite:absolutesize: readonly vector2 (The size of the sprite multiplied by the scale to get the size of the sprite on screen)

Render

Contains general rendering values.

```
render.text = bool

render.graphics = bool

render.priority = number (changes order)

render.textbg = colour

render.graphicsbg = colour

render.textcolour = colour

render.vram() (Current vram usage and the total amount of vram
installed as a tuple (usage, capacity))
```

Sound

The computer has a 5-channel sound chip.

- 1-2: Square wave, you can set the duty to make it a pulse wave
- 3: Triangle wave
- 4: Noise
- 5: Sample, you can set this to any ROBLOX audio that is below 3.5s in duration.

`sound.Play(channel, note, volume, duty)`

Notes can be either a frequency (hz) or a string. Providing a blank note value will end the note playing in the channel.

```
sound.Play(1, 440, 1, 2) -- play 440hz on channel 1
sound.Play(3, "c3") -- play note c3 on channel 3
wait(2)
sound.Play(1) -- stop on channel 1
sound.Play(3) -- stop on channel 3
```

`sound.Mod(channel, note, volume, duty)`

This function is like the “play” function, except that it will not start any new notes. All arguments are optional except for channel.

```
sound.play(1, 220)
for i=1, 220 do
    sound.Mod(1, 220+i) -- change pitch
end
sound.Mod(1, nil, nil, 3) -- change duty
```

`sound.SetSample(sample, object)`

Sets the sound to be played in channel 5. Sample must be a ROBLOX sound asset ID and must be under 3.5 seconds in duration. The sound will not loop if oneshot is set to true

```
set.SetSample(123456789)
sound.play(5, 440, 1)
```

OS

os.clock() (Amount of time since the computer has turned on)

os.gettime() (Ingame time returned as a tuple (S,M,H))

os.getclock() (Same as above, but returned as a string (H:M))

os.cls() (Clear text output)

os.makefile(filename: string, contents: string)

os.readfile(filename: string) (Returns the contents of a file and returns ‘false’ if the file is not found on the system)

GPIO

GPIO expansions enable the computer to send and receive signals. GPIO expansions must be connected to the expansion slot on the back of the computer. Ports are numerically addressed closest to furthest. The output is red and the input is blue.

`gpio.Power(port_index:number, voltage:number?, colour:colour?)`

```
while wait(.1) do
gpio.power(1, nil, 0x20)
wait(.1)
gpio.power(2, nil, 0xb0)
end
```

`gpio:Connect(port_index:number, callback:function(voltage:number?, colour, Color3))`

```
gpio:Connect(1, function()
    print("input 1")
    beep(440, 0.2)
end)
gpio:Connect(2, function()
    print("input 2")
    beep(523, 0.2)
end)
```

Voltage is currently unused and is for forwards compatibility with the building overhaul.

Control

Both `control[1]` and `control[2]` are tables with the current input state for the given controller. It will set to false ifno controller is connected to the port indexed.

`control:Connect(controller_portnumber, callback:function(input_type:string, pressed/direction:bool | vector2, keycode: Enum.KeyCode))`

This will fire upon up and down button presses, and each time the joystick is moved. Keyboard input is on port 0.

The valid input types are:

- ButtonA
- ButtonB
- ButtonSelect
- ButtonStart
- Joystick
- Keyboard

Input handler example:

```
local joystick_direction = vector2.zero
control:Connect(1, function(input_type, value))
    if input_type == "Joystick" then
        joystick_direction = value
        print(joystick_direction)
    elseif input_type == "ButtonA" then
        if value == true then
            print("pressed button a!")
        else
            print("unpressed button a!")
        end
    end
end)
```